1.8 Iterators and Generators

In Section 1.4.2, we introduced the for-loop syntax beginning as:

for *element* **in** *iterable*:

and we noted that there are many types of objects in Python that qualify as being iterable. Basic container types, such as list, tuple, and set, qualify as iterable types. Furthermore, a string can produce an iteration of its characters, a dictionary can produce an iteration of its keys, and a file can produce an iteration of its lines. User-defined types may also support iteration. In Python, the mechanism for iteration is based upon the following conventions:

- An ***iterator*** is an object that manages an iteration through a series of values. If variable, i, identifies an iterator object, then each call to the built-in function, next(i), produces a subsequent element from the underlying series, with a StopIteration exception raised to indicate that there are no further elements.
- An ***iterable*** is an object, obj, that produces an *iterator* via the syntax iter(obj).

By these definitions, an instance of a list is an iterable, but not itself an iterator. With data = [1, 2, 4, 8], it is not legal to call next(data). However, an iterator object can be produced with syntax, i = iter(data), and then each subsequent call to next(i) will return an element of that list. The for-loop syntax in Python simply automates this process, creating an iterator for the give iterable, and then repeatedly calling for the next element until catching the StopIteration exception.

More generally, it is possible to create multiple iterators based upon the same iterable object, with each iterator maintaining its own state of progress. However, iterators typically maintain their state with indirect reference back to the original collection of elements. For example, calling iter(data) on a list instance produces an instance of the list_iterator class. That iterator does not store its own copy of the list of elements. Instead, it maintains a current *index* into the original list, representing the next element to be reported. Therefore, if the contents of the original list are modified after the iterator is constructed, but before the iteration is complete, the iterator will be reporting the *updated* contents of the list.

Python also supports functions and classes that produce an implicit iterable series of values, that is, without constructing a data structure to store all of its values at once. For example, the call range(1000000) does *not* return a list of numbers; it returns a range object that is iterable. This object generates the million values one at a time, and only as needed. Such a ***lazy evaluation*** technique has great advantage. In the case of range, it allows a loop of the form, **for** j **in** range(1000000):, to execute without setting aside memory for storing one million values. Also, if such a loop were to be interrupted in some fashion, no time will have been spent computing unused values of the range.

We see lazy evaluation used in many of Python's libraries. For example, the dictionary class supports methods keys( ), values( ), and items( ), which respectively produce a "view" of all keys, values, or (key,value) pairs within a dictionary. None of these methods produces an explicit list of results. Instead, the views that are produced are iterable objects based upon the actual contents of the dictionary. An explicit list of values from such an iteration can be immediately constructed by calling the list class constructor with the iteration as a parameter. For example, the syntax list(range(1000)) produces a list instance with values from 0 to 999, while the syntax list(d.values( )) produces a list that has elements based upon the current values of dictionary d. We can similarly construct a tuple or set instance based upon a given iterable.

Generators

In Section 2.3.4, we will explain how to define a class whose instances serve as iterators. However, the most convenient technique for creating iterators in Python is through the use of ***generators***. A generator is implemented with a syntax that is very similar to a function, but instead of returning values, a **yield** statement is executed to indicate each element of the series. As an example, consider the goal of determining all factors of a positive integer. For example, the number 100 has factors 1, 2, 4, 5, 10, 20, 25, 50, 100. A traditional function might produce and return a list containing all factors, implemented as:

```
def factors(n):                    # traditional function that computes factors
  results = [ ]                    # store factors in a new list
  for k in range(1,n+1):
    if n % k == 0:                 # divides evenly, thus k is a factor
      results.append(k)            # add k to the list of factors
  return results                   # return the entire list
```

In contrast, an implementation of a *generator* for computing those factors could be implemented as follows:

```
def factors(n):                    # generator that computes factors
  for k in range(1,n+1):
    if n % k == 0:                 # divides evenly, thus k is a factor
      yield k                      # yield this factor as next result
```

Notice use of the keyword **yield** rather than **return** to indicate a result. This indicates to Python that we are defining a generator, rather than a traditional function. It is illegal to combine yield and return statements in the same implementation, other than a zero-argument return statement to cause a generator to end its execution. If a programmer writes a loop such as **for** factor **in** factors(100):, an instance of our generator is created. For each iteration of the loop, Python executes our procedure

until a yield statement indicates the next value. At that point, the procedure is temporarily interrupted, only to be resumed when another value is requested. When the flow of control naturally reaches the end of our procedure (or a zero-argument return statement), a StopIteration exception is automatically raised. Although this particular example uses a single yield statement in the source code, a generator can rely on multiple yield statements in different constructs, with the generated series determined by the natural flow of control. For example, we can greatly improve the efficiency of our generator for computing factors of a number, n, by only testing values up to the square root of that number, while reporting the factor $n//k$ that is associated with each k (unless $n//k$ equals k). We might implement such a generator as follows:

```
def factors(n):                # generator that computes factors
  k = 1
  while k * k < n:             # while k < sqrt(n)
    if n % k == 0:
      yield k
      yield n // k
    k += 1
  if k * k == n:               # special case if n is perfect square
    yield k
```

We should note that this generator differs from our first version in that the factors are not generated in strictly increasing order. For example, factors(100) generates the series $1, 100, 2, 50, 4, 25, 5, 20, 10$.

In closing, we wish to emphasize the benefits of lazy evaluation when using a generator rather than a traditional function. The results are only computed if requested, and the entire series need not reside in memory at one time. In fact, a generator can effectively produce an infinite series of values. As an example, the Fibonacci numbers form a classic mathematical sequence, starting with value 0, then value 1, and then each subsequent value being the sum of the two preceding values. Hence, the Fibonacci series begins as: $0, 1, 1, 2, 3, 5, 8, 13, \ldots$. The following generator produces this infinite series.

```
def fibonacci( ):
  a = 0
  b = 1
  while True:                  # keep going...
    yield a                    # report value, a, during this pass
    future = a + b
    a = b                      # this will be next value reported
    b = future                 # and subsequently this
```